MERN Practice Questions

Q1. Develop a Student Management REST API using NodeJS, ExpressJS, MongoDB, and Mongoose.

Create an ExpressJS application, connect it to MongoDB using Mongoose, and define a Student schema with the following fields: name, email, semester, specialization, and cgpa.

Implement the following tasks:

1. Create Student: Implement POST /students to accept student details in JSON format and store the student in MongoDB using Mongoose.

2. View All Students: Implement GET /students to retrieve and display all students stored in the database.

3. View Student by ID: Implement GET /students/:id to retrieve a particular student using their MongoDB ID.

4. Update Student: Implement PUT /students/:id to update the details of an existing student using their MongoDB ID.

5. Delete Student: Implement DELETE /students/:id to delete an existing student using their MongoDB ID.

Demonstrate all five APIs using Postman or Thunder Client and show the request as well as the corresponding JSON response for each API.

Q2.Develop a Book Management REST API using NodeJS, ExpressJS, MongoDB, and Mongoose.

Create an ExpressJS application, connect it to MongoDB using Mongoose, and define a Book schema with the following fields: title, author, category, price, and publishedYear.

Implement the following tasks:

1. Add Book: Implement POST /books to store a new book in MongoDB.

2. View All Books: Implement GET /books to retrieve all books.

3. View Book by ID: Implement GET /books/:id to retrieve a book using its MongoDB ID.

4. Update Book: Implement PUT /books/:id to update an existing book.

5. Delete Book: Implement DELETE /books/:id to delete a book.

Demonstrate all five APIs using Postman or Thunder Client with appropriate JSON requests and responses.

Q3. Develop an Employee Management REST API using NodeJS, ExpressJS, MongoDB, and Mongoose.

Create an ExpressJS application, connect it to MongoDB using Mongoose, and define an Employee schema with the following fields: name, email, department, designation, and salary.

Implement the following tasks:

1. Create Employee: Implement POST /employees to store employee details in MongoDB.
2. View All Employees: Implement GET /employees to retrieve all employees.
3. View Employee by ID: Implement GET /employees/:id to retrieve an employee using their MongoDB ID.
4. Update Employee: Implement PUT /employees/:id to update employee details.
5. Delete Employee: Implement DELETE /employees/:id to delete an employee.

Demonstrate all five APIs using Postman or Thunder Client and show the JSON request and response.

Q4. Develop a Patient Management REST API using NodeJS, ExpressJS, MongoDB, and Mongoose.

Create an ExpressJS application, connect it to MongoDB using Mongoose, and define a Patient schema with the following fields: name, age, gender, disease, and doctor.

Implement the following tasks:

1. Register Patient: Implement POST /patients to store patient details in MongoDB.
2. View All Patients: Implement GET /patients to retrieve all patients.
3. View Patient by ID: Implement GET /patients/:id to retrieve a particular patient.
4. Update Patient: Implement PUT /patients/:id to update patient details.
5. Delete Patient: Implement DELETE /patients/:id to delete a patient record.

Demonstrate all five APIs using Postman or Thunder Client and display the JSON request and response.

Q5. Develop an Event Management REST API using NodeJS, ExpressJS, MongoDB, and Mongoose.

Create an ExpressJS application, connect it to MongoDB using Mongoose, and define an Event schema with the following fields: eventName, date, venue, organizer, and capacity.

Implement the following tasks:

1. Create Event: Implement POST /events to store a new event in MongoDB.
2. View All Events: Implement GET /events to retrieve all events.
3. View Event by ID: Implement GET /events/:id to retrieve a particular event.
4. Update Event: Implement PUT /events/:id to update event details.
5. Delete Event: Implement DELETE /events/:id to delete an event.

Demonstrate all five APIs using Postman or Thunder Client with appropriate JSON requests and responses.

Q6. Develop a Hotel Room Management REST API using NodeJS, ExpressJS, MongoDB, and Mongoose.

Create an ExpressJS application, connect it to MongoDB using Mongoose, and define a Room schema with the following fields: roomNumber, roomType, price, floor, and isAvailable.

Implement the following tasks:

1. Add Room: Implement POST /rooms to store room details in MongoDB.
2. View All Rooms: Implement GET /rooms to retrieve all rooms.
3. View Room by ID: Implement GET /rooms/:id to retrieve a particular room.
4. Update Room: Implement PUT /rooms/:id to update room details.
5. Delete Room: Implement DELETE /rooms/:id to delete a room.

Demonstrate all five APIs using Postman or Thunder Client and show the JSON request and response for each API.

==========

Q1. Write a NodeJS program using the fs module to manage product information.

1. Create a file products.txt and store details of at least 5 products.
2. Read and display the product details.
3. Append details of a new product.
4. Check whether the file exists before performing the read operation.
5. Display an appropriate message for a missing file or operation error.

Q2. Write a NodeJS program using the fs module to perform basic file operations.

1. Create a file notes.txt and write 5 lines of text into it.
2. Read and display the file contents.
3. Append 2 additional lines to the file.
4. Rename the file to my_notes.txt.
5. Display an appropriate message if any file operation fails.

Q3. Create a NodeJS application using a user-defined module named result.js.
The module should contain functions to calculate total marks, percentage, and grade.

1. Create and export the required functions from result.js.
2. Import the module into nodeapp.js.
3. Define marks for five subjects in nodeapp.js and display the total, percentage, and grade.
4. Handle invalid marks appropriately.

Q4. Create a NodeJS application using a user-defined module named area.js.
The module should contain functions to calculate the area of a circle, rectangle, and triangle.

1. Create and export the three functions from area.js.
2. Import the module into nodeapp.js. — 3 marks

3. Define suitable dimensions in nodeapp.js and display the area of all three shapes.
4. Handle invalid dimensions appropriately.

Q5. Create a NodeJS application using a user-defined module named numberUtils.js.
The module should contain functions to check whether a number is even/odd, positive/negative.

1. Create and export the required functions from numberUtils.js.
2. Import the module into nodeapp.js.
3. Define a number in nodeapp.js and display the result of both checks.
4. Handle invalid number values appropriately.